



Projeto e Análise de Algoritmos

Abordagens para Resolução
de Problemas

André Tavares da Silva
andre.silva@udesc.br

Abordagens para Resolução de Problemas

Muitos problemas podem ser resolvidos de diversas maneiras diferentes.

Contudo, dependendo da abordagem escolhida, uma solução pode ser melhor ou pior do que outra em questões de tempo ou uso de memória.

Abordagens para Resolução de Problemas

Podemos classificar os principais métodos de resolução de problemas em relação a abordagem utilizada:

- Indução Matemática – Fórmula
- Divisão e Conquista (*divide and conquer*)
- Algoritmos Gulosos (*greedy*)
- Algoritmos de Tentativa e Erro (*backtracking*)
- Programação dinâmica (*dynamic programming*)
- Algoritmos de aproximação (*approximation*)

Indução Matemática – Fórmula

Abordagem mais simples possível.

O problema pode ser reduzido a uma fórmula ou a um conjunto de fórmulas matemáticas.

Exemplo: descobrir as raízes de uma equação de segundo grau.

Algoritmos de Divisão-e-Conquista

(Divide and Conquer)

Já estudamos esta abordagem.

Desmembrar o problema original em vários subproblemas semelhantes (menores), resolver os subproblemas (executando o mesmo processo recursivamente) e combinar as soluções.

Exemplos: ordenar um vetor de elementos (*Merge-Sort*, *Quick-Sort*) e multiplicação entre inteiros grandes

Algoritmos Gulosos

(Greedy Algorithms)

Uma técnica para resolver problemas de otimização: minimizar ou maximizar um determinado valor.

Um algoritmo guloso sempre faz a escolha que parece ser a melhor em um determinado momento, esperando que essa melhor escolha local leve ao melhor resultado do problema como um todo.

Uma técnica simples, mas que na maioria das vezes não leva ao resultado ótimo. Quando leva, a prova não costuma ser trivial.

Algoritmos Gulosos

(Greedy Algorithms)

Um algoritmo guloso (*greedy algorithm*) constrói a solução para um problema sempre fazendo as **escolhas que parecem mais vantajosas** naquele momento.

Um algoritmo guloso é um algoritmo considerado míope porque toma decisões **com base apenas nas informações disponíveis no momento**, sem considerar as consequências que essas decisões terão no futuro.

Ele **nunca se arrepende** e não desfaz as escolhas já feitas.

Algoritmos Gulosos

(Greedy Algorithms)

Alguns problemas que podem ser resolvidos com algoritmos gulosos:

- Problema do melhor troco - canônico
- Problema da Árvore Geradora Mínima
- Escalonamento de tarefas
- Codificação de Huffman
- Problema da Mochila Fracionada

Problema do Melhor Troco - canônico

Considere que o sistema econômico de um local utiliza unidades monetárias de valor: $M_0, M_1, M_2, \dots, M_n$;

Qual é o menor número de unidades monetárias que representam um determinado valor V ?

Exemplo: Moedas disponíveis: 1, 5, 10, 25, 50, 100

Valor do troco: 3.82

Qual a menor quantidade de moedas a serem entregues e quais são elas?

Problema do Melhor Troco - canônico

Para sistemas monetários **canônicos** como os usualmente utilizados, podemos adotar a abordagem de escolher o maior número possível das maiores moedas:

Exemplo: Moedas disponíveis: 1, 5, 10, 25, 50, 100

Valor do troco: \$ 3.82

$382 / 100 = 3$ e sobram \$ 82

3 moedas de 100

$82 / 50 = 1$ e sobram 32

1 moeda de 50

$32 / 25 = 1$ e sobram 7

1 moeda de 25

$7 / 5 = 1$ e sobram 2

1 moeda de 5

$2 / 1 = 2$ e sobram 0

2 moedas de 1

Problema do Melhor Troco - canônico

CUIDADO! Para sistemas monetários **não canônicos** a abordagem apresentada pode não resultar no melhor troco possível:

Exemplo Trivial: Moedas disponíveis: 1, 3, 4
Valor do troco: \$ 0.06

Qual é o número mínimo de moedas para o troco?

Problema do Melhor Troco - canônico

Quando o algoritmo guloso falha no problema do troco?

Quando existem moedas x, y tal que $x < y$ e $2x > y$

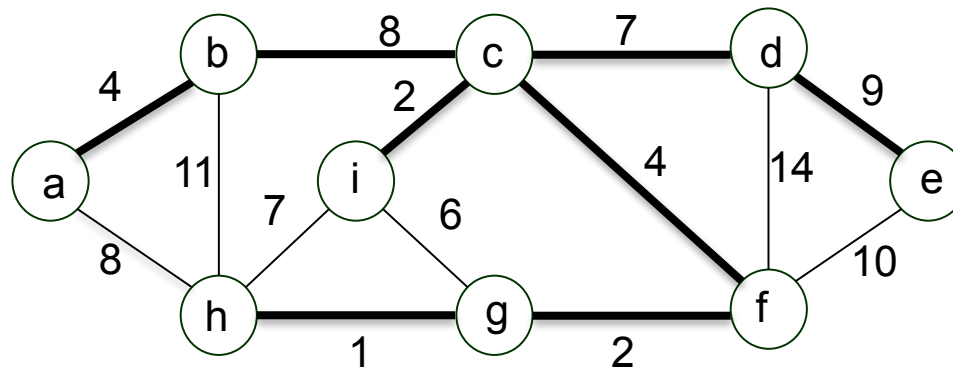
Exemplo: Moedas disponíveis: 2, 10, 20, 50, 100, 200 OK

Exemplo: Moedas disponíveis: 1, 5, 8 Falha

Problema da Árvore Geradora Mínima

Dado um grafo com seus respectivos vértices e arestas, determinar qual a árvore que contém todos os vértices deste grafo com o menor custo possível.

Escolhemos sempre a melhor aresta dos nós já incluídos que não formem um circuito (Prim)



Escalonamento de Tarefas

Dada uma lista de tarefas a serem executadas com um horário de início e um horário de término, determinar qual a quantidade máxima de atividades que podem ser executadas

Exemplo: Um auditório só pode ser utilizado para um evento por vez. Em um dia com muitos eventos, deseja-se determinar qual é o maior número de eventos que podem ser realizados no auditório, e quais são eles (OBS: pode haver mais de uma solução).

Evento	1	2	3	4	5	6	7	8	9	10	11
Início	3	8	5	1	6	12	0	8	5	2	3
término	5	11	7	4	10	14	6	12	9	13	8

Escalonamento de Tarefas

Solução gulosa: ordenamos os eventos pelo horário de término (em ordem crescente) e sempre que possível pegamos o evento com menor horário de término.

Evento	4	1	7	3	11	9	5	2	8	10	6
Início	1	3	0	5	3	5	6	8	8	2	12
término	4	5	6	7	8	9	10	11	12	13	14

Codificação de Huffman

Um método de compressão de dados. Esse método usa a frequência com que cada símbolo ocorre, em uma coleção de dados, para determinar um código de tamanho variável para o símbolo.

Caractere	Frequência	Código (fixo)	Código (variável)
a	45	000	0
b	13	001	101
c	12	010	100
d	16	011	111
e	9	100	1101
f	5	101	1100

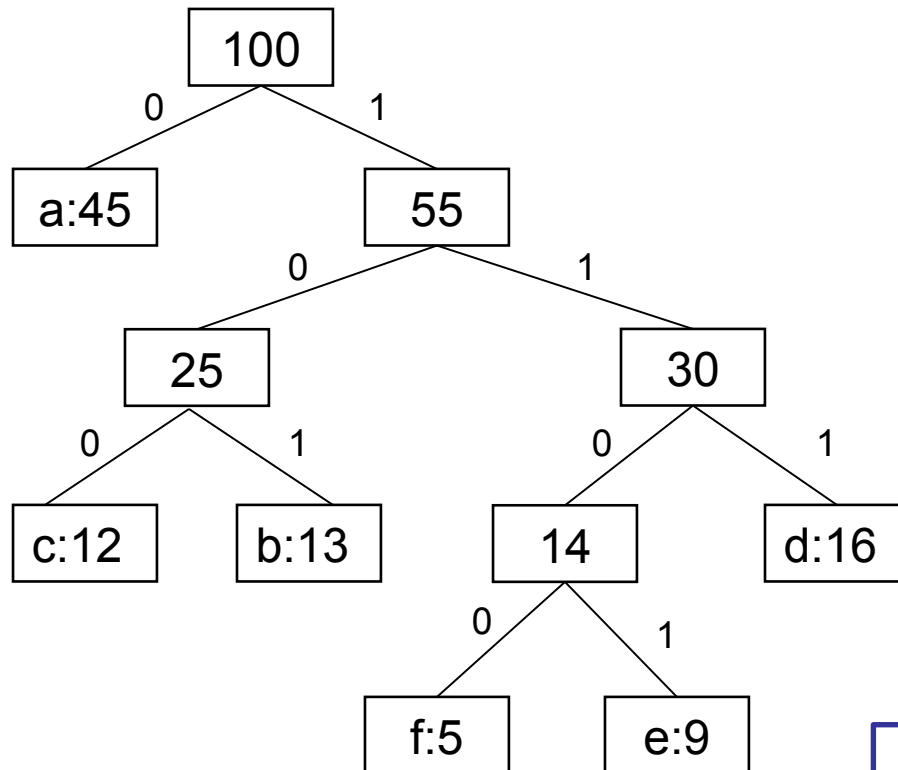
Exemplo:

... faca ...

... 101000010000 ...

... 110001000 ...

Codificação de Huffman



Exemplo:

... faca ...

... 101000010000 ...

... 110001000 ...

Codificamos usando a notação:

Filhos a esquerda: 0

Filhos a direita: 1

Até chegar a letra desejada

Codificação de Huffman

Huffman(C)

$n \leftarrow |C|$

$Q \leftarrow C$

para $i \leftarrow 1$ **até** $n - 1$

$z \leftarrow \text{AlocaNo}()$

$x \leftarrow z.\text{esq} \leftarrow \text{Extrair-Minimo}(Q)$

$y \leftarrow z.\text{dir} \leftarrow \text{Extrair-Minimo}(Q)$

$z.\text{valor} = x.\text{valor} + y.\text{valor}$

Inserir(Q, z)

retorne Q

Testando o algoritmo:

Como ficaria a codificação para a string:

“testando o teste”

Construir a árvore e o mapa de codificação

Importante:

Note que para decodificar uma string, é necessário conhecer a árvore que foi usada para gerá-la!
Ou seja, espaço utilizado: nova string (menor) + árvore

Problema da Mochila Fracionada

Dark Version: Um ladrão está assaltando uma loja e possui uma mochila que pode carregar até P kg sem arrebentar. Sabendo os pesos e valores dos itens, qual é o maior valor possível que o ladrão conseguirá roubar?

Na versão da mochila fracionada os elementos podem ser pegos em fração (apenas parte do objeto, com proporção linear entre peso e valor)

Exemplo: Mochila com capacidade 9 kg

Objetos	1	2	3	4	5	6
Peso	1	3	2	5	7	9
Valor	3	2	4	7	9	12

Problema da Mochila Fracionada

Solução Gulosa: Ordenamos o vetor de itens em relação a proporção: [valor / peso]

Objetos	1	3	4	6	5	2
Peso	1	2	5	9	7	3
Valor	3	4	7	12	9	2
Proporção	3	2	1.4	1.33	1.29	0.67

Tentativa e Erro

(Backtracking)

Outra técnica para resolver problemas de otimização

Testa todas as possíveis soluções até encontrar a(s) melhor(es) solução(ões) para um problema, utilizando as ideias de busca em profundidade ou busca em largura

Pode utilizar validações para diminuir (de maneira não muito significativa) o escopo de busca [ex: desconsiderar soluções parciais piores do que uma solução final já encontrada]

Geralmente não adequada para instâncias muito grandes

Tentativa e Erro (*Backtracking*)

Alguns problemas clássicos que são resolvidos (também) com *backtracking* (instâncias pequenas):

- Problema do passeio do cavalo
- Puzzle 3x3
- Sokoban
- Caixeiro Viajante

Passeio do Cavalo

Partindo de uma posição inicial, qual é o menor número de movimentos que um cavalo deve realizar para chegar a qualquer uma das casas de um tabuleiro?



LEMBRETE: o cavalo (*knight*) anda apenas em movimentos 'L'

Exemplo: tab. 5x5 $c \rightarrow 2,1$

- Busca em Profundidade...
- Busca em Largura...

Tentativa e Erro (*Backtracking*)

Solução usando Busca em Profundidade:

[+] usa menos memória (apenas tabuleiro)

[-] repete muitos movimentos (mais lento)

Solução usando Busca em Largura:

[+] não repete movimentos (mais rápido)

[-] usa mais memória (para salvar as posições)
em alguns casos se torna inviável

Puzzle 3x3

Montar a imagem original reorganizando as peças, podendo movimentar as peças usando a única lacuna.



Como resolver
computacionalmente?

Puzzle 3x3

Como resolver computacionalmente?

- Abstrair que os movimentos se baseiam na lacuna: ela pode ir para cima, baixo, direita e esquerda;
- Representar o ‘puzzle’ como uma estrutura de dados;
- Armazenar a quantidade de movimentos (mínima) necessárias para alcançar uma dada posição.
- Descobrir o mínimo para chegar à posição final.



Puzzle 3x3

Exemplo:

Peças são representadas por números entre 1 e 8

Lacuna é representada pelo número 0

1	0	3
4	2	5
7	8	6

Representação decimal/string: 103 425 786

Transições: 013 425 786 / 130 425 786 / 123 405 786

Sokoban

Levar as caixas para pontos objetivos, em qualquer ordem, com o menor número de movimentos possível.

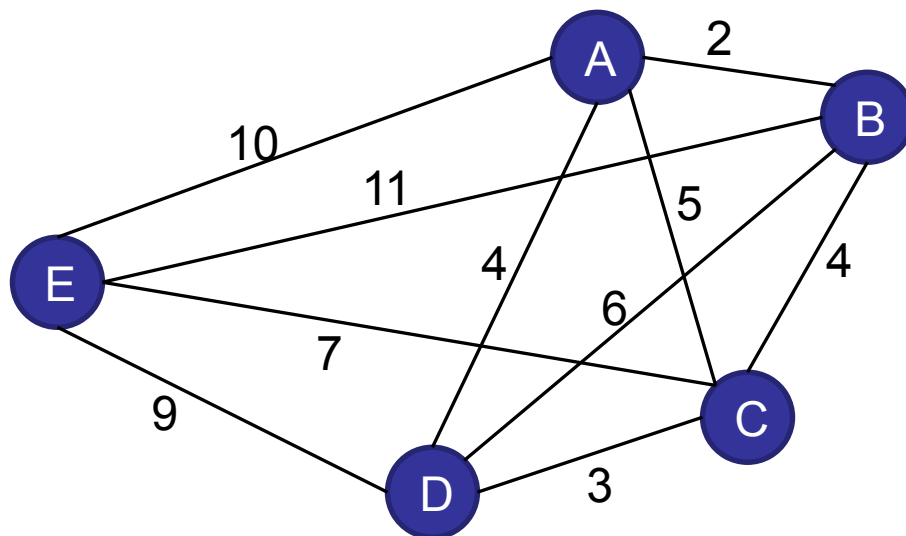


Quais valores representam um estado neste problema?

Problema do Caixeiro Viajante

Um vendedor deseja visitar n cidades e retornar a cidade de origem. Dado um grafo não orientado completo com n vértices, onde existe um custo $c(i, j)$ (associado a cada aresta) para viajar da cidade i a cidade j .

Qual o trajeto com custo mínimo?



Exercícios

Escreva como seria codificada a sequência de caracteres: “Abracadabra pe de cabra” utilizando a codificação de Huffman. Mostre também a árvore criada no processo de codificação.

(LEMBRETE: os espaços em branco também são caracteres)

Escreva um programa que informa qual o número mínimo de jogadas para resolver o seguinte “puzzle3x3”

1	0	3
4	2	5
7	8	6

[Linguagens: C/C++/C#/JAVA/Python/Haskell]

Programação Dinâmica

(Dynamic Programming)

Uma abordagem auxiliar para reduzir a computação em problemas de otimização.

Consiste em dividir o problema original em subproblemas mais simples, resolvê-los, armazenar os resultados em memória e consultar estes resultados posteriormente a fim de reduzir o espaço de busca.

Resultados são calculados apenas uma vez (nunca são recalculados por outra iteração)

Programação Dinâmica

(Dynamic Programming)

Alguns problemas que podem ser resolvidos com programação dinâmica:

- Fibonacci
- Crivo de Eratostenes
- Problema do Menor Troco não canônico
- Problema da Mochila Binária (0/1)
- Maior Subsequência Comum
- Multiplicação de Cadeias de Matrizes
- Sokoban

Fibonacci

O modelo recursivo de Fibonacci pode ser extremamente ruim dependendo da implementação:

```
int fib(int n) {  
    if( n<=1 ) return n;  
    return fib(n-1) + fib(n-2);  
}
```

Fibonacci

Para entender a programação dinâmica, talvez o problema da sequência de Fibonacci seja o exemplo mais simples para demonstrar o seu uso.

A ideia é utilizar uma estrutura auxiliar para armazenar os resultados e utilizá-los posteriormente

Fibonacci

Abordagem *top-down* (*Memoization*)

```
Fib( $n$ )  
  se  $n \leq 1$  então  
    retorne  $n$   
  senão  
    se F[n] está indefinido  
      F[n]  $\leftarrow$  Fib( $n - 1$ ) + Fib( $n - 2$ )  
    retorne F[n]
```

O vetor **F** é inicializado antes da chamada da função com um valor padronizado que nunca será registrado [ex: -1]

Fibonacci

Abordagem *bottom-up*

Fib(n)

F[0] = 0

F[1] = 1

para $i \leftarrow 2$ até n

F[i] = $F[i - 2] + F[i - 1]$

retorne **F[n]**

Crivo de Eratosthenes

(Sieve of Eratosthenes)

O Crivo de Eratóstenes é um algoritmo clássico e altamente eficiente para encontrar todos os números primos até um limite inteiro N .

Ele funciona criando uma lista de 2 a N e, iterativamente, "riscando" (marcando) todos os múltiplos dos números primos encontrados até que restem apenas os primos.

Crivo de Eratosthenes

(Sieve of Eratosthenes)

```
void crivoEratosthenes(int n) {  
    // inicializa todos como true (potencialmente primos)  
    bool primo[n + 1];  
    for (int i = 0; i <= n; i++) primo[i] = true;  
  
    // Otimização: basta verificar até a raiz quadrada de n  
    int limite = sqrt(n);  
    for (int p = 2; p <= limite; p++) {  
        if (primo[p] == true) {  
            // Atualiza todos os múltiplos de p para false  
            for (int i = p * p; i <= n; i += p) {  
                primo[i] = false;  
            }  
        }  
    }  
}
```

Crivo de Eratosthenes

(Sieve of Eratosthenes)

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100
101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120

Prime numbers

Problema do Melhor Troco não canônico

Como visto anteriormente, para sistemas monetários **não canônicos** a abordagem gulosa pode não resultar no melhor resultado!

Nesse caso, podemos usar programação dinâmica como um método eficiente para encontrar o melhor troco:

Exemplo Trivial: Moedas disponíveis: 1, 3, 4

Valor do troco: \$ 0.06

Qual é o número mínimo de moedas para o troco?

Problema do Melhor Troco não canônico

Exemplo Trivial: Moedas disponíveis: 1, 3, 4

Valor do troco: \$ 0.06

Considere as moedas como um vetor: $M = \{ 1, 3, 4 \}$

Inicialmente assumimos que o troco para \$ 0.00 precisa de zero moedas

$$T(0) = 0$$

E seguimos iterativamente até o troco desejado usando a fórmula:

$$T(n) = \min(1+T(n-M[0]), 1+T(n-M[1]), 1+T(n-M[2]))$$

Problema do Melhor Troco não canônico

Exemplo Trivial: Moedas disponíveis: 1, 3, 4

Valor do troco: \$ 0.06

$$T(0) = 0$$

$$T(n) = \min(1+T(n-M[0]), 1+T(n-M[1]), 1+T(n-M[2]))$$

$$T(1) = \min(1+T(1-1), 1+\cancel{T(1-3)}, 1+\cancel{T(1-4)}) = 1$$

$$T(2) = \min(1+T(2-1), 1+\cancel{T(2-3)}, 1+\cancel{T(2-4)}) = 2$$

$$T(3) = \min(1+T(3-1), 1+T(3-3), 1+\cancel{T(3-4)}) = 1$$

(...)

Complexidade de tempo $\Rightarrow \Theta (n * |M|)$

Complexidade de espaço $\Rightarrow \Theta (n)$

Problema da Mochila Binária (0/1)

Dark Version: Um ladrão está assaltando uma loja e possui uma mochila que pode carregar até P kg sem arrebentar. Sabendo os pesos e valores dos itens, qual é o maior valor possível que o ladrão conseguirá roubar?

- Na versão da mochila binária os elementos só podem ser pegos como um todo (não podemos pegar metade de um item – ou pega, ou não pega).
- **Exemplo:** Mochila com capacidade 9 kg

Objetos	1	2	3	4	5	6
Peso	1	3	2	5	7	9
Valor	3	2	4	7	9	12

Problema da Mochila Binária (0/1)

Solução Matricial

Montamos uma matriz para armazenar as melhores soluções intermediárias e as usamos para gerar as próximas soluções;

Objetos	1	2	3	4	5	6
Peso	1	3	2	5	7	9
Valor	3	2	4	7	9	12

Problema da Mochila Binária (0/1)

Solução Matricial

Objetos	1	2	3	4	5	6
Peso	1	3	2	5	7	9
Valor	3	2	4	7	9	12

Item / Capacidade	0	1	2	3	4	5	6	7	8	9
1	0	3	3	3	3	3	3	3	3	3
2	0	3	3	3	5	5	5	5	5	5
3	0	3	4	7	7	7	9	9	9	9
4	0	3	4	7	7	7	10	11	14	14
5	0	3	4	7	7	7	10	11	14	14
6	0	3	4	7	7	7	10	11	14	14

$$m[item][k] = \max(m[item-1][k], \text{valor} + m[item-1][k-\text{peso}])$$

Problema da Mochila Binária (0/1)

Solução Matricial

Complexidade de tempo $\Rightarrow \Theta(\text{peso} * \text{qtd-itens})$

Complexidade de espaço $\Rightarrow \Theta(\text{peso} * \text{qtd-itens})$

Maior Subsequência Comum

(Longest Common Subsequence)

Dadas duas strings S e T , qual é maior subsequencia (sequencia não necessariamente contígua), da esquerda para a direita, entre estas strings?

- **Exemplo:** $S = \text{ABAZDC}$
 $T = \text{BACBAD}$

Resposta ?

Resolução similar a abordagem matricial para resolver o problema da mochila.

Maior Subsequência Comum

(Longest Common Subsequence)

Solução Matricial

pA / pB		A	B	A	Z	D	C
	0	0	0	0	0	0	0
B	0						
A	0						
C	0						
B	0						
A	0						
D	0						

1 - Preenche linha e coluna fictícia

2 - usar regra:

$$m[i][j] = \max(m[i-1][j], m[i][j-1])$$

SE $pA[i] == pB[j]$:

$$m[i][j] = \max(m[i][j], m[i-1][j-1] + 1)$$

Maior Subsequência Comum

(Longest Common Subsequence)

Solução Matricial

pA / pB		A	B	A	Z	D	C
	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1
A	0	1	1	2	2	2	2
C	0	1	1	2	2	2	3
B	0	1	2	2	2	2	3
A	0	1	2	3	3	3	3
D	0	1	2	3	3	4	4

Multiplicação de Cadeias de Matrizes

(Matrix Chain Multiplication)

Dadas uma sequência de n matrizes, qual é a melhor ordem para multiplicá-las a fim de realizar menos multiplicações.

OBS: a multiplicação entre matrizes é associativa, logo
 $(AB)(CD) = A(B(CD)) = (AB)C)D = (A(BC))D = A((BC)D)$

- **Exemplo:** São dadas 4 matrizes com os seguintes tamanhos:

$$M_1 = (2,3)$$

$$M_1 * M_2 = 2 * 3 * 6 = 36$$

$$M_2 = (3,6)$$

$$M_2 * M_3 = 3 * 6 * 4 = 72$$

$$M_3 = (6,4)$$

$$M_3 * M_4 = 6 * 4 * 5 = 120$$

$$M_4 = (4,5)$$

Multiplicação de Cadeias de Matrizes

(Matrix Chain Multiplication)

- Exemplo:** São dadas 4 matrizes com os seguintes tamanhos:

$$M_1 = (2,3)$$

$$M_1 * M_2 = 2 * 3 * 6 = 36$$

$$M_2 = (3,6)$$

$$M_2 * M_3 = 3 * 6 * 4 = 72$$

$$M_3 = (6,4)$$

$$M_3 * M_4 = 6 * 4 * 5 = 120$$

$$M_4 = (4,5)$$

MATRIZ	1	2	3	4
1	0	36		
2		0	72	
3			0	120
4				0

MATRIZ	1	2	3	4
1	0	1		
2		0	2	
3			0	3
4				0

Multiplicação de Cadeias de Matrizes

(Matrix Chain Multiplication)

- Exemplo:** São dadas 4 matrizes com os seguintes tamanhos:

$$M_1 = (2,3) \quad (M_1 * M_2) M_3 = 36 + 2 * 6 * 4 = 84$$

$$M_2 = (3,6) \quad M_1 (M_2 * M_3) = 2 * 3 * 4 + 72 = 96$$

$$M_3 = (6,4) \quad (M_2 * M_3) M_4 = 72 + 3 * 4 * 5 = 132$$

$$M_4 = (4,5) \quad M_2 (M_3 * M_4) = 3 * 6 * 5 + 120 = 210$$

MATRIZ	1	2	3	4
1	0	36	84	
2		0	72	132
3			0	120
4				0

MATRIZ	1	2	3	4
1	0	1	2	
2		0	2	3
3			0	3
4				0

Multiplicação de Cadeias de Matrizes

(Matrix Chain Multiplication)

- Exemplo:** São dadas 4 matrizes com os seguintes tamanhos:

$$M_1 = (2,3) \quad (M_1 M_2 M_3) M_4 = 84 + 2 * 4 * 5 = 124$$

$$M_2 = (3,6) \quad (M_1 M_2) * (M_3 M_4) = 36 + 2 * 6 * 5 + 120 = 216$$

$$M_3 = (6,4) \quad M_1 (M_2 M_3 M_4) = 2 * 3 * 5 + 132 = 162$$

$$M_4 = (4,5)$$

MATRIZ	1	2	3	4
1	0	36	84	124
2		0	72	132
3			0	120
4				0

MATRIZ	1	2	3	4
1	0	1	2	3
2		0	2	3
3			0	3
4				0

Multiplicação de Cadeias de Matrizes

(Matrix Chain Multiplication)

PSEUDOCÓDIGO

```
MATRIX-CHAIN-MULT( v[n] )  
  para i de 1 a n:                                // zera a primeira linha  
    m[ i , i ] = 0  
  para c de 2 a n:                                  // para todas as linhas posteriores  
    para i de 1 a (n - c + 1):                      // para todos os elem. da diagonal  
      j = i + c - 1  
      m[ i , j ] =  $\infty$   
      para k de i a (j - 1):                        // para todas as permutações  
        q = m[ i , k ] + m[ k+1 , j ] + v[ i-1 ] * v[ k ] * v[ j ]  
        se q < m[ i , j ]:  
          m[ i , j ] = q  
          s[ i , j ] = k
```

Multiplicação de Cadeias de Matrizes

(Matrix Chain Multiplication)

PSEUDOCÓDIGO

```

PRINT-EQUACAO ( s[n][n] , i , j )
    se i == j
        print "M" + i
    senão
        print "("
        PRINT-EQUACAO( s , i , s[i , j] )
        PRINT-EQUACAO( s , s[i , j] + 1 , j )
        print ")"
    
```

MATRIZ	1	2	3	4
1	0	1	2	3
2		0	2	3
3			0	3
4				0

```

P-E (1,4)
  P-E(1,3)
    P-E(1,2)
      P-E(1,1) => "M1"
      P-E(2,2) => "M2"
    P-E(3,3) => "M3"
  P-E(4,4) => "M4"

  ( M1 M2 ) M3 ) M4 )
  
```

Sokoban

Abordagem usando programação dinâmica

Posição do jogador (x,y) | Posição das caixas (x,y)

(1,2) | (2,2) $\Rightarrow$ [1][2][2][2] = 0 (movimentos)



Algoritmos de Aproximação

(Approximation Algorithms)

Existem alguns problemas os quais não se conhece uma solução eficiente. Dizemos que estes problemas são “difíceis” ou intratáveis pois, para instâncias grandes, o tempo de processamento seria inviável.

Nestas situações é comum remover a exigência de procurar pela solução ótima e passamos a procurar por uma solução próxima da ótima (solução boa / razoável)

Algoritmos de Aproximação

(Approximation Algorithms)

Existem duas abordagens principais:

- **Heurística/Metaheurísticas:** algoritmo que buscam soluções próximas a solução ótima. Utilizam alguma informação (ou intuição) sobre a instância do problema para resolvê-lo de forma eficiente. Podem ser genéricas (servem p/ vários problemas).
- **Algoritmo Aproximado:** algoritmo que resolve um problema de forma eficiente e ainda garante a qualidade da solução. É necessário provar que a solução está próxima da ótima. Geralmente únicos para cada problema.

- **Algoritmos Bio-inspirados:** algoritmo baseados no comportamento de seres vivos:
 - **Algoritmos genéticos:** gera soluções, escolhe as melhores (seleção natural) e gera um novo ciclo, oferecendo espaço para mutações (alterações nos componentes das soluções).
 - **Colônia de formigas:** modela a geração de soluções baseado no comportamento das formigas com comunicação através do ambiente (como elas sempre encontram um doce?!).
 - **Enxame de partículas:** se baseia em princípios da revoada de pássaros com influência local (histórico de cada partícula) e global/social (histórico geral)

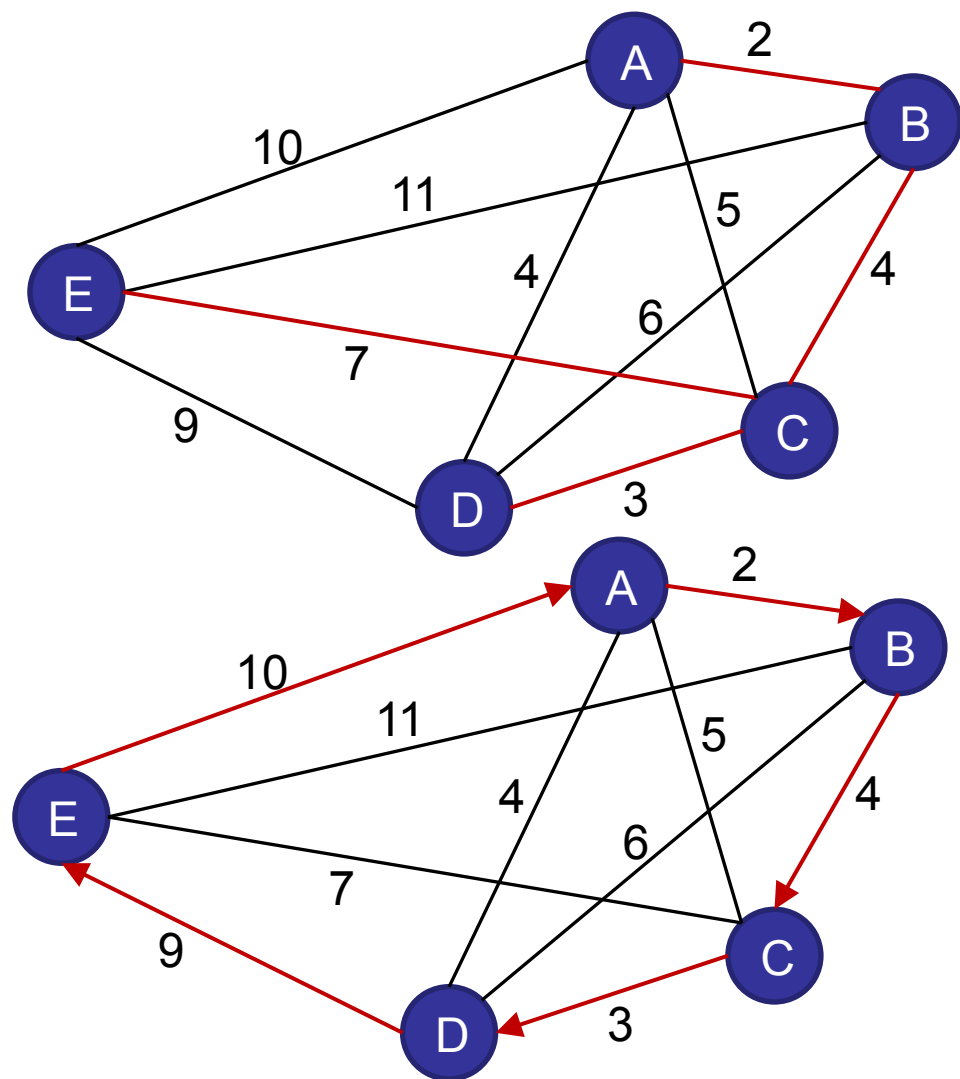
Caixeiro Viajante

Dado o grafo $G = (V, A)$ e o custo c :

1. Selecione um vértice $r \in V$ para ser o vértice *raiz*.
2. Obtenha a árvore geradora mínima a partir de r .
3. Faça H ser a lista de vertices ordenados de acordo com a primeira visita, considerando um percurso em pré-ordem, retornando a r .

Se a função custo satisfaz a desigualdade de triângulos: $c(u, w) < c(u, v) + c(v, w)$

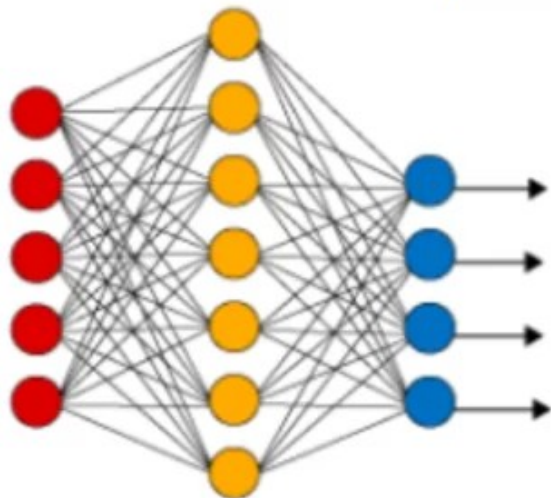
$$c(H) < 2c(\text{ótimo})$$



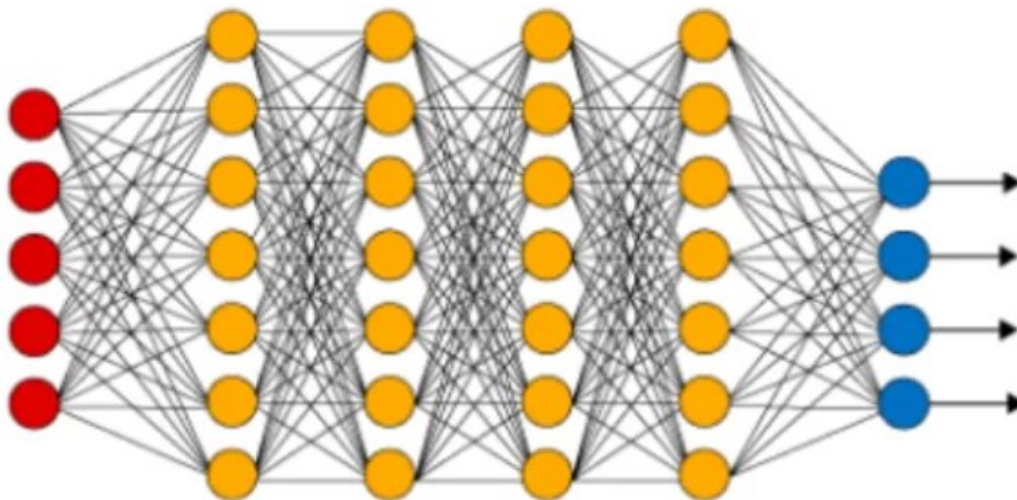
Algoritmos Aproximados

Redes neurais operam como aproximadores universais de funções e são computacionalmente intensivas. Encontrar a melhor solução é inviável. Portanto, elas utilizam algoritmos de aproximação, como a retropropagação (Backpropagation).

Rede Neural Simples



Rede Neural Profunda



● Camada de entrada ● Camada escondida ● Camada de saída

Exercícios

- O pequeno Turing exagerou na quantidade de disciplinas neste semestre. Para passar nas provas finais ele precisa estudar uma quantidade de tempo para cada disciplina. Se ele estudar o tempo necessário com certeza será aprovado, caso contrário, certamente reprovará. Cada disciplina tem a sua importância subjetiva (i_s) para o menino, sendo que ele quer maximizar este índice de importância uma vez que sabe que não tem tempo viável para estudar para todas as disciplinas. Dada a lista de disciplinas e sua importância subjetiva, descubra qual o melhor cronograma de estudo para Turing a fim de maximizar a importância subjetiva (i_s) na aprovação (mostre a tabela utilizada na solução).

- Tempo disponível: 16 horas
- Disciplinas:
 - ALP (3h – 5is);
 - CAL (6h – 6is);
 - REC (9h – 5is);
 - TEC (8h – 6is);
 - PAP (5h – 3is);
 - BAN (4h – 4is);
 - PPR (3h – 2is);
 - SOP (4h – 4is);

Exercícios

- Considere as strings *genoma* e *feature* que representam, respectivamente, uma parte do genoma de uma criatura e uma determinada característica de interesse. Deseja-se descobrir qual é a maior subsequência comum (sequencia de caracteres iguais, mas não necessariamente contíguas) entre essas duas cadeias de caracteres:

genoma: ACCUGTATAUCGUCACTU

feature: GCAUTTC